COMP 110 Final Exam Study Guide

Chapters 6 to 11

Name _____ Date _____

Use these 72 questions to explain concepts, trace code, repair errors, and design tested Java methods and classes. Each chapter has twelve questions. Try the questions by hand before compiling a test version; use separate paper for longer programs.

Scope and assumptions. Review Chapters 6 through 11: methods, arrays, multidimensional arrays, objects and classes, thinking in objects, and inheritance and polymorphism. Earlier arithmetic, selection, and loop skills remain prerequisites. Target Java 17. Unless stated otherwise, snippets belong inside main, array and String inputs are non-null, and needed imports are available. Class and method declarations are shown at class scope.

How to study. For traces, draw references and record exact values and output. For algorithms, state preconditions, the result, and why the loop stops. Test ordinary, boundary, empty, and invalid cases as permitted by each contract. These practice questions do not define the exact exam contents.

Chapter 6 Methods

Identify signatures, trace calls, explain pass by value, design contracts, resolve overloads, and write reusable methods.

1. Identify the return type, method name, formal parameters, and signature of public static double average(int total, int count). Write a call and explain how arguments differ from parameters.

Response _____

_____

2. Trace the call. Explain why the value in main changes or stays the same.

```
static void change(int n) { n = 99; }
public static void main(String[] args) {
    int n = 5;
    change(n);
    System.out.println(n);
}
```

Response _____

_____

3. Decide whether each overload pair is legal: f(int) and f(double); int g(int) and double g(int); h(int, double) and h(double, int). For the last pair, explain h(1, 2).

Response _____

_____

4. Repair this method, preserving a non-void return type. Its contract requires a nonempty array. Explain the missing-return problem.

```
static int maximum(int[] a) {
    for (int x : a)
        if (x > 0) return x;
}
```

Response _____

_____

5. Explain scope and lifetime for a method parameter, a block-local variable, and a static field. May two separate methods each declare a local variable named total?

Response _____

_____

6. Write static boolean isPrime(int n) for every int. Return false below 2. Avoid an overflow-prone divisor * divisor comparison. Give tests for negative input, 0, 1, 2, a square, and a larger prime.

Response _____

_____

_____

7. Write static int count(String s) that counts ASCII letters A-Z and a-z. Assume s is non-null. Include a mixed-input test and an empty-string test. Explain how this contract differs from all Unicode letters.

Response _____

_____

_____

8. Write static boolean isPalindrome(String s) for non-null ASCII text. Compare exact case and include spaces and punctuation. Test empty text, one character, noon, and Noon.

Response _____

_____

_____

9. Write static char randomLowercase() using Math.random. Explain both endpoints. How would you produce uppercase letters? Give a property test rather than one expected random sequence.

Response _____

_____

10. For static void printRange(int start, int end), specify a clear contract before implementing it. Require start <= end and small enough values to avoid overflow. Explain why printing and returning are different.

Response _____

_____

11. Trace the overload selection. Explain which decisions depend on declared argument types.

```
static String label(int n) { return "int"; }
static String label(double n) { return "double"; }
public static void main(String[] args) {
    System.out.println(label(2));
    System.out.println(label(2.0));
}
```

Response

12. Decompose a score-report program into methods for reading data, calculating a result, and displaying it. Why should a calculation method usually avoid reading Scanner input or printing?

Response

Chapter 7 Single Dimensional Arrays

Trace aliases, copy and traverse arrays, count and summarize data, reverse, search, sort, and handle command line arguments.

13. For double[] list = {3.4, 2.0, 3.5, 5.5}, give list[1], list.length, and the last valid index. Explain why list[4] fails. Compare new int[3] with an uninitialized local array reference.

Response

14. Trace the aliasing and output. Draw the two references after list2 = list1. Ignore the final trailing space in your answer.

```
int[] list1 = {3, 2, 1};
int[] list2 = {1, 2, 3};
list2 = list1;
for (int i = 0; i < list1.length; i++)
    list1[i] = list1[i] * list2[i];
for (int i = list2.length - 1; i >= 0; i--)
    System.out.print(list2[i] + " ");
```

Response

15. Explain array assignment, clone(), Arrays.copyOf, and System.arraycopy. For a primitive int array, show an independent copy. What changes for an array of mutable objects?

Response

16. Write static int count(int[] list) that counts every odd occurrence, including duplicates and negative odd numbers. Assume a non-null array. Test {5,10,15,5,30,43,5,60,70,80}, {-3,0,2}, and empty input.

Response

17. Write static boolean isSorted(int[] list) for nondecreasing order. Assume non-null input; duplicates, empty arrays, and one-element arrays are allowed. Test {1,1,3}, {3,1}, and empty input.

Response

18. Write static double average(int[] list), requiring a nonempty array and at most one million elements. Use long for the sum. Explain why dividing two ints before assigning to double loses information.

Response

19. Write static void reverse(int[] list) in place for a non-null array. State the time cost and test even, odd, empty, and one-element arrays.

Response

20. Write static int linearSearch(int[] list, int key). Return the first matching index, or -1. Does the array need to be sorted? Test duplicate and absent keys.

Response

21. Write binarySearch(int[] list, int key) for an ascending sorted array. Return any matching index or -1. Trace key 7 in {1,3,5,7,9}. Explain the precondition and cost.

Response

22. Write ascending selectionSort(int[] list). After each pass, identify the sorted part. Trace {3,1,2}. Compare its worst-case growth with binary search.

Response

23. Explain why for (int x : list) { x = 0; } does not zero the array. Show an indexed repair. Explain mutation versus reassignment when an array is passed to a method.

Response

24. Given java Test + 3 abc 2, list args.length and args[2]. Explain whether the class name belongs to args, and give a guard before accessing args[2].

Response

Chapter 8 Multidimensional Arrays

Reason about rectangular and ragged shapes, nested traversal, row sharing, copying, totals, and matrix algorithms.

25. For int[][] a = {{1,2,3},{4},{}};, give a.length, every row length, and the sum of all row lengths. Which expressions are valid: a[0][2], a[1][1], a[2].length?

Response _____

_____

26. Compare new int[2][3] and new int[2][]. State which entries are initialized to zero and which are null. What must happen before assigning a[0][0] in the second form?

Response _____

_____

27. Trace all output. Explain why each inner loop uses the current row length.

```
int[][] a = {{1,2},{3},{}};
for (int r = 0; r < a.length; r++) {
    int sum = 0;
    for (int c = 0; c < a[r].length; c++)
        sum += a[r][c];
    System.out.println(sum);
}
```

Response _____

_____

28. Write static long total(int[][] a) for non-null outer and row arrays, allowing ragged and empty rows. Assume at most one million int elements. Give a test with a negative value and an empty row.

Response _____

_____

_____

29. For a rectangular 2 by 3 matrix {{2,4,6},{1,3,5}}, calculate the row totals and column totals. Outline loops for each. How would you define column totals for ragged rows?

Response _____

_____

30. Find the row with the largest total in {{-4,-1},{-2,-2},{}}. Empty rows total zero. Return the first row on a tie; a zero-row outer array returns -1. Explain the all-negative pitfall.

Response _____

_____

31. Trace the copy. Does clone() copy each row? Show how to obtain independent primitive rows.

```
int[][] a = {{1,2},{3,4}};
int[][] b = a.clone();
b[0][0] = 99;
System.out.println(a[0][0]);
System.out.println(a == b);
```

Response _____

32. Write transpose(int[][] a) for a non-null rectangular matrix with at least one non-null row and at least one column. Return a new matrix. State the result for {{1,2,3},{4,5,6}}.

Response _____

33. For the square matrix {{1,2,3},{4,5,6},{7,8,9}}, find the main diagonal sum and the other diagonal sum. Give the indexes for each. If summing both diagonals together with each cell counted once, what changes?

Response _____

34. A grade table stores one student per row and one question per column. Given answers {{'A','B','C'},{'A','C','C'}} and key {'A','B','C'}, calculate each score. State a shape contract and outline the comparison loops.

Response _____

35. A 3 by 3 tic-tac-toe board uses X, O, or a space. Describe how to detect a winning row, column, or diagonal. Why is testing only three equal cells insufficient?

Response _____

36. Explain what can change when a matrix is passed to a method: a cell, a row reference, or the parameter itself. Give one test that distinguishes row replacement from copying a row.

Response _____

Chapter 9 Objects and Classes

Design and construct objects, protect valid state, distinguish static and instance members, and trace references.

37. Distinguish a class, an object, an instance field, a method, and a reference variable. Which describes object state? Does declaring Circle c create a Circle?

Response _____

38. Explain constructors, an explicitly written no-argument constructor, and the compiler-provided default constructor. If class Circle declares only Circle(double radius), is new Circle() legal?

Response _____

_____

39. Compare default values of int, boolean, and reference instance fields with initialization of local variables. Can a class method access a private field through an instance of its own class?

Response _____

_____

40. Trace shared and per-object state. Explain every output value.

```
static class Counter {
    static int count;
    int value;
    Counter() { count++; value++; }
}
public static void main(String[] args) {
    Counter a = new Counter(), b = new Counter();
    System.out.println(a.count + " " + b.count + " "
        + a.value + " " + b.value);
}
```

Response _____

_____

41. Design a Circle class with private radius, a constructor, getRadius, setRadius, and getArea. Require a finite nonnegative radius. Show how this.radius resolves a parameter name collision.

Response _____

_____

_____

42. Trace aliasing. Explain why == and an object content comparison serve different purposes.

```
static class Box { int value; }
public static void main(String[] args) {
    Box a = new Box();
    Box b = a;
    b.value = 8;
    System.out.println(a.value);
    System.out.println(a == b);
}
```

Response _____

_____

43. Explain mutation and reassignment in this object-parameter example. Predict the value printed in main.

```
static class Box { int value; }
static void update(Box b) {
    b.value = 7;
    b = new Box();
    b.value = 99;
}
public static void main(String[] args) {
    Box b = new Box();
    update(b);
    System.out.println(b.value);
}
```

Response _____

_____

44. For Circle[] circles = new Circle[3], how many Circle objects are constructed? Explain the initial element values and show how to initialize all elements with radius 1.

Response _____

_____

45. Convert these UML members to Java declarations: - balance : double; + Account(initial : double); + getBalance() : double; + deposit(amount : double) : void. State a useful deposit contract.

Response _____

_____

46. A static main method attempts to call an instance method getArea() with no object. Explain the compile error and give a repair. When is static appropriate?

Response _____

_____

47. Explain why getters and setters do not automatically provide good encapsulation. Give an invariant and a test that checks a rejected update preserves it.

Response _____

_____

48. An immutable Profile holds a name String and an int[] of scores. Is a final score-array reference sufficient for immutability? Explain safe constructor and getter behavior.

Response _____

_____

Chapter 10 Thinking in Objects

Use abstraction and composition, design Money Course and stack contracts, and work with wrappers, text, and regular expressions.

49. Distinguish abstraction and encapsulation. Explain an is-a relationship and a has-a relationship using Course, Student, Circle, and Cylinder.

Response _____

50. A Money class stores nonnegative integer dollars and cents from 0 to 99. Explain an invariant and why integer cents help. Compute $3.75 + $2.50 and $3.75 - $5.00 under the chapter contract that negative subtraction returns null.

Response _____

51. Write addCents(int dollars, int cents, int extraCents) returning a long total cent count. Assume nonnegative values, cents <= 99. Explain the placement of the cast and test a large dollar amount.

Response _____

52. Design a Tank that has a current amount and a capacity. Both are finite, capacity is positive, and amount stays between zero and capacity. Specify fill and drain behavior before choosing fields. Should an invalid request partially modify state?

Response _____

53. A Course stores Student references in a private array and a count. Explain the difference between capacity and logical size. What must addStudent and getStudents guarantee? Does copying the array copy Student objects?

Response _____

54. Outline a StackOfIntegers using a private int array and a size. Give push, pop, peek, and isEmpty contracts, including full and empty states. Trace push(4), push(9), peek(), pop(), pop().

Response _____

55. Compare int, Integer, autoboxing, and unboxing. Trace the code. Explain the risk of unboxing null and why wrapper == is a poor value-equality test.

```
Integer boxed = Integer.valueOf(7);
int value = boxed;
System.out.println(value + 1);
```

Response _____

56. Compare Integer.parseInt("25"), Integer.valueOf("25"), and Double.parseDouble("5.6"). Which returns a primitive? Is Integer.parseInt("5.6") valid input? State a parsing contract.

Response _____

_____

57. Trace the String and StringBuilder code. Explain which object is mutable and whether snapshot changes.

```
String s = "Java";
s.concat("!");
StringBuilder b = new StringBuilder(s);
b.append("!");
String snapshot = b.toString();
b.append("?");
System.out.println(s);
System.out.println(snapshot);
System.out.println(b);
```

Response _____

_____

58. Evaluate "123".matches("\\d+") and "x123".matches("\\d+"). Explain full-string matching and why a Java string needs two backslashes for a regex backslash. Give a regex for exactly three ASCII digits.

Response _____

_____

59. Predict the effect of "a,b;c".split("[,;]"). Compare replace with replaceAll. How do you split on a literal dot safely?

Response _____

_____

60. A Length class could store centimeters internally while exposing inches. Which decisions belong in its public contract? Explain one test that would allow the internal representation to change safely.

Response _____

_____

Chapter 11 Inheritance and Polymorphism

Trace constructor chains and dispatch, apply access rules, cast safely, define equality, and use lists and stacks.

61. Explain extends, inherited accessible methods, private superclass fields, and constructors. Does a subclass object still contain superclass state even when that state is private?

Response _____

_____

62. Trace construction under Java 17. If A had only A(int n), what would B() need to do?

```
static class A { A() { System.out.println("A"); } }
static class B extends A {
    B() { System.out.println("B"); }
}
public static void main(String[] args) { new B(); }
```

Response _____

63. Distinguish overriding, overloading, static method hiding, and private methods. Why is @Override useful? May an override reduce public access to protected?

Response _____

64. Trace dynamic dispatch. What changes if move is static? Explain declared type versus runtime type.

```
static class Vehicle {
    public String move() { return "Vehicle"; }
}
static class Car extends Vehicle {
    @Override public String move() { return "Car"; }
}
public static void main(String[] args) {
    Vehicle v = new Car();
    System.out.println(v.move());
}
```

Response _____

65. Given Object obj = new SimpleCar() and SimpleCar declares public void drive(int miles), why is obj.drive() invalid? Show a guarded call that drives 10 miles. What if obj is null?

Response _____

66. Explain access for public, protected, package-private, and private members. Across packages, can any client access a protected member? State the important subclass qualifier restriction.

Response _____

67. Repair the declaration and identify the forbidden access. Assume the two classes are in one package. Why may Car call setID but not assign ID directly?

```
class Vehicle {
    protected String name;
    private int ID;
    public setID(int value) { ID = value; }
}
class Car extends Vehicle {
    Car() { setID(99); ID = 47; name = "Honda"; }
}
```

Response _____

_____

68. Compare ==, default Object.equals, and a value-class equals(Object) override. State the equals contract and the matching hashCode requirement. Why is equals(Circle other) alone insufficient?

Response _____

_____

69. Trace ArrayList<Integer> operations. Distinguish removal by index from removal by value. Give the valid get indexes after the final operation.

```
java.util.ArrayList<Integer> a = new java.util.ArrayList<>();
a.add(1); a.add(7); a.add(2);
a.remove(1);
System.out.println(a);
a.remove(Integer.valueOf(2));
System.out.println(a);
```

Response _____

_____

70. For ArrayList<Object> containing "Greetings", new Object(), and a SimpleCar whose toString returns "I drive fast", describe the three printed lines. Is the default Object suffix a specified memory address?

Response _____

_____

71. A MyStack uses private ArrayList<Object> elements. Write pop assuming empty input must throw NoSuchElementException. Explain why size-1 is the top index. Compare peek and pop, and explain why composition protects its API.

Response _____

_____

_____

72. Explain final on a variable, method, and class. If final ArrayList<String> names = new ArrayList<>(), may names.add("Ada") run? Discuss one integrated test plan for Course membership or a stack.

Response _____

_____